

Detailed steps

Refer : JWT Details n diagrams

To understand why JWT n what is it ?

For JWT details : <https://jwt.io/introduction>

Refer to debugger n introduction sections

Detailed steps :

0. Copy earlier project created for Basic Authentication

1. Add jjwt dependencies in pom.xml

```
<!-- https://mvnrepository.com/artifact/io.jsonwebtoken/jjwt-api -->
```

```
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.11.5</version>
</dependency>
```

```
<!-- https://mvnrepository.com/artifact/io.jsonwebtoken/jjwt-impl -->
```

```
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-impl</artifactId>
  <version>0.11.5</version>
  <scope>runtime</scope>
</dependency>
```

```
<!-- https://mvnrepository.com/artifact/io.jsonwebtoken/jjwt-jackson -->
```

```
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-jackson</artifactId>
  <version>0.11.5</version>
  <scope>runtime</scope>
</dependency>
```

2. Add these properties in application.properties

#JWT properties

#JWT Secret key for signing n Verification

SECRET_KEY=mySecretKey1234

#JWT expiration timeout in msec : 24*3600*1000

EXP_TIMEOUT=86400000

(Better approach -

Refer : <https://generate-random.org/encryption-key-generator>) : for generating random keys

3. Create JWTUtils class .Add @Component for enabling it's D.I in the controller later.

Methods in JWTUtils

3.1 generate JWT token

3.2 get user name/email from the token

3.3 validate token

4.To Intercept all incoming requests (except those mentioned with permit All in sec config class), create a custom filter class : extending from `org.springframework.web.filter.OncePerRequestFilter`. Spring security will call its `doFilterInternal` method , once per request

Steps

4.1 Extract JWT from the request header

Get Authorization header from request & check if it is not null & starting with "Bearer "

4.2 If it's present , extract & validate it (using JWT utils validate method : to check if it's not been tampered !)

4.3 In case of valid token , extract claims(user name / email & authorities)

extract user name or email and authorities from the token (using JWT Utils methods)

4.4 Create Authentication object(class : `UsernamePasswordAuthenticationToken`)

`public UsernamePasswordAuthenticationToken(Object principal, Object credentials, GrantedAuthority[] authorities)`

principal : user name / email extracted from JWT

credentials : null (DO NOT store the credentials / pwd in the token)

authorities : extracted from the token

4.5 Save this authentication token in the sec ctx.

`SecurityContextHolder.getContext().setAuthentication(authentication);`

5. Configure JWT in security configuration class.

5.1 NO changes since Basic Auth , i.e no method needed for authentication

5.2 Modify authorization rules

Summary :

1. enable cors & disable CSRF

`http.cors().and().csrf().disable()`

2. In addition to other patterns : allow any HTTP OPTIONS request (which is typically to allow a pre flight request coming from react like front end . NOTE : it's not required for testing it with postman

`antMatchers(HttpMethod.OPTIONS, "**").permitAll()`

3. Configure session management policy

To tell Spring Security NEVER create an `HttpSession` & DO NOT use `HttpSession` to obtain the `SecurityContext`

`sessionManagement().sessionCreationPolicy(SessionCreationPolicy.STATELESS);`

4. Add custom jwt filter before a filter for processing an authentication based upon form submission.

`http.addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);`

5.3 In our authentication REST controller , we are going to need to access the authentication manager.

By default, it's not publicly accessible, and we need to configure Auth Manager bean

@Bean

`public AuthenticationManager authenticationManager(AuthenticationConfiguration config) throws`

`Exception {`

`return config.getAuthenticationManager();`

`}`

6. Create a class - `CustomAuthenticationEntryPoint` implements `AuthenticationEntryPoint`

Reason - When a client makes an unauthenticated request to a resource that they are not authorized to access, in this case, an implementation of `AuthenticationEntryPoint` is used to request credentials from the client.

The AuthenticationEntryPoint implementation may perform a redirect to a log in page OR respond with an WWW-Authenticate header, or simply send HTTP response error code 401 , implying that this end point need HTTP authentication.

7. Create AuthController : for signin n signup

For signup (user registration)

7.1 Request payload : User DTO (containing : user details +UserRole : enum)

7.2 Using User Service layer , store user details (with encoded password n enum of roles mapped to Role entities) in DB .

8. Dependencies of UserSignInSignUpController : Authentication Manager n JWTUtils

Steps in authentication

For signin

/*

* Add new end point in UserController , for signin

* URL - http://host:port/users/signin

* Method - POST

* Request payload : Auth req DTO : email n password

* Resp payload : In case of success : Auth Resp DTO : msg + JWT token + SC 200

* In case of failure : SC 401

*/

8.1

Create UsernamePasswordAuth token to wrap ONLY user supplied credentials(em n pass) , no authorities n authenticated=false

API : UsernamePasswordAuthenticationToken(String userName,String password)

8.2 Invoke Auth mgr's authenticate method , by passing above auth token

It rets : fully populated Authentication object (including granted authorities)if successful.

In case of failure :

Throws :

DisabledException : if an account is disabled

LockedException : if an account is locked

BadCredentialsException : if incorrect credentials are presented by client

8.3 In case of success : return

ResponseEntity : SC OK

Resp DTO : Auth successful msg & generate JWT using utils n add it .

-----More Details-----

For JWT details : <https://jwt.io/introduction>

Refer to debugger n introduction sections

For spring security javadocs : <https://docs.spring.io/spring-security/site/docs/current/api/>

HttpSecurity is a builder class and provides numerous convenience methods that can be chained. Under the hood, each method adds a filter the HTTP request needs to pass through

3.1 API : `org.springframework.security.core.Authentication` i/f

Methods : `getAuthorities` , `getPrincipal` , `getCredentials`

3.2 Authentication i/f method

`java.lang.Object getPrincipal()`

The identity of the principal being authenticated. In the case of an authentication request with username and password, this would be the username.

Since we have added these user details in the customized implementation of `o.s.s.c.userdetails.UserDetails` , you can type cast it to `CustomUserDetailsImpl`.

3.3 `o.s.s.c.userdetails.UserDetails` : i/f

Provides core user information.

You have to implement it to store user information which is later encapsulated into Authentication object.

3.4 `org.springframework.security.authentication.AuthenticationManager`

Implementing Classes: `ProviderManager`

Method :

`Authentication authenticate(Authentication authentication)` throws `AuthenticationException`

Attempts to authenticate the passed Authentication object, returning a fully populated Authentication object (including granted authorities) if successful.

An AuthenticationManager must honour the following contract concerning exceptions:

A `DisabledException` must be thrown if an account is disabled and the AuthenticationManager can test for this state.

A `LockedException` must be thrown if an account is locked and the AuthenticationManager can test for account locking.

A `BadCredentialsException` must be thrown if incorrect credentials are presented. While the above exceptions are optional, an AuthenticationManager must always test credentials.

Exceptions should be tested for and if applicable thrown in the order expressed above (i.e. if an account is disabled or locked, the authentication request is immediately rejected and the credentials testing process is not performed). This prevents credentials being tested against disabled or locked accounts.

Parameters:

`authentication` - the authentication request object(credentials)

Returns:

a fully authenticated object including credentials (principal)

Throws:

`AuthenticationException` - if authentication fails

3.5

`org.springframework.security.authentication.UsernamePasswordAuthenticationToken`

An imple class of Authentication i/f which is designed for simple presentation of a username and password.

Simply said it's the holder of username n password (i.e user credentials)